

Índice

2. Vulnerabilidades e Ataques
- 3.1. Funções de Digest (Hash Criptográfico)
- 3.2. Derivação de Chaves (KDF)
- 3.3. Criptografia Simétrica Moderna
- 3.4. Criptografia Assimétrica
- 3.5. Message Authentication Codes (MAC)
- 3.6. Assinaturas Digitais
4. Public Key Infrastructure (PKI)

2. Vulnerabilidades e Ataques

Esta seção estabelece os conceitos fundamentais de segurança: o que são vulnerabilidades, como são exploradas e quais os impactos. É a base para entender por que precisamos de todos os mecanismos criptográficos que vêm a seguir.

Sumário de Termos

Vulnerability (Vulnerabilidade) Conceito

Fraqueza num sistema que pode ser explorada para comprometer segurança.

Threat (Ameaça) Conceito

Potencial causa de incidente indesejado que pode resultar em dano a sistemas ou dados.

Attack (Ataque) Conceito

Tentativa deliberada de explorar vulnerabilidades para comprometer segurança.

Impact (Impacto) Conceito

Consequência negativa resultante de exploração bem-sucedida de vulnerabilidade.

CWE (Common Weakness Enumeration) Standard

Taxonomia hierárquica de tipos de vulnerabilidades de software organizada em classes, bases, variantes e compostas.

Buffer Overflow Vulnerabilidade

Vulnerabilidade onde dados excedem limites de memória alocada, potencialmente sobrescrevendo dados adjacentes.

Stack-based Buffer Overflow Vulnerabilidade

Buffer overflow específico na stack que pode sobrescrever endereços de retorno de funções permitindo execução de código arbitrário.

Out-of-bounds Access Vulnerabilidade

Acesso a posições de memória fora dos limites válidos de estrutura de dados.

Rainbow Table Attack Ataque

Ataque usando tabelas pré-computadas de hashes para reverter hashes de passwords rapidamente.

Brute Force Attack Ataque

Tentativa sistemática de todas as combinações possíveis até encontrar a correta.

Side-channel Attack Ataque

Ataque que explora informação vazada por implementação física (timing, consumo energético, radiação eletromagnética).

Timing Attack Ataque

Side-channel que analisa tempo de execução para inferir informação secreta através de variações temporais.

Malware Ataque

Software malicioso desenhado para danificar, explorar ou comprometer sistemas.

Ransomware Ataque

Malware que cifra dados da vítima e exige resgate para descriptografar.

Exploit Ataque

Código ou técnica que aproveita vulnerabilidade específica para obter acesso não autorizado ou executar ações maliciosas.

🔗 Fundamentos Essenciais (20%)

Relação fundamental: Vulnerabilidades existem em sistemas → Ameaças (threats) representam potencial de exploração → Ataques exploram ativamente as vulnerabilidades → Impactos são as consequências. A cibersegurança trabalha em minimizar vulnerabilidades, mitigar ameaças, detetar ataques e reduzir impactos.

CWE como mapa: Entender a hierarquia CWE ajuda a ver padrões. Por exemplo: CWE-787 (Out-of-bounds Write) é filho de CWE-119 (Improper Restriction of Operations within Memory Buffer) que é filho de CWE-118 (Incorrect Access of Indexable Resource). Isto mostra como vulnerabilidades específicas derivam de falhas mais gerais.

Defense in depth: Nunca confies numa única camada de segurança. Buffer overflows são prevenidos por: validação de input, uso de linguagens memory-safe, DEP (Data Execution Prevention), ASLR (Address Space Layout Randomization), stack canaries. Se uma falhar, outras protegem.

Princípio crítico: A maioria das vulnerabilidades resulta de não validar inputs. Sempre assume que qualquer input externo é malicioso até provares o contrário. Valida comprimento, tipo, formato e range antes de usar.

3.1. Funções de Digest (Hash Criptográfico)

Hashes são a fundação de quase toda a criptografia moderna. São usados em assinaturas digitais, MACs, KDFs, blockchain, verificação de integridade e muito mais. Entender hashes é essencial para todo o resto.

Sumário de Termos

Digest/Hash Function (Função de Hash) Conceito

Função que transforma dados de tamanho arbitrário num valor fixo (digest) de forma unidirecional e determinística.

Collision Resistance (Resistência a Colisões) Propriedade

Propriedade que garante ser computacionalmente inviável encontrar dois inputs diferentes que produzam o mesmo hash.

Avalanche Effect (Efeito Avalanche) Propriedade

Propriedade onde uma pequena alteração no input causa mudança drástica e imprevisível no output (idealmente 50% dos bits mudam).

SHA-256/SHA-512 Algoritmo

Funções de hash da família SHA-2 que produzem digests de 256 e 512 bits respetivamente, consideradas seguras atualmente.

MD5 Algoritmo (Legado)

Função de hash legada (128 bits) considerada insegura devido a vulnerabilidades de colisão descobertas.

Compression Function (Função de Compressão) Componente

Componente interno que processa blocos de dados iterativamente com um IV (Initialization Vector) para construir o hash final.

🔗 Fundamentos Essenciais (20%)

Conceito central: Hashes são "impressões digitais" de dados - qualquer alteração mínima no input muda completamente o output (**avalanche effect**), mas o **mesmo input produz sempre o mesmo hash**. São unidirecionais (não podes recuperar o original) e detetam qualquer modificação em dados.

Três propriedades essenciais: 🔒

1. **Pré-imagem:** Dado hash H, é impossível encontrar mensagem M tal que hash(M) = H

2. **Segunda pré-imagem:** Dado M1, é impossível encontrar M2 ≠ M1 tal que hash(M1) = hash(M2)

3. **Resistência a colisões:** É impossível encontrar qualquer par (M1, M2) onde M1 ≠ M2 mas hash(M1) = hash(M2)

Aplicação prática: Verificação de integridade de ficheiros (downloads comparam hash publicado), armazenamento de passwords (nunca guardes passwords em texto claro, guarda o hash!), blockchain (cada bloco contém hash do anterior), e base para MACs e assinaturas digitais.

Limitação crítica: Hashes sozinhos não impedem modificação intencional - um atacante pode alterar dados E calcular novo hash. Para autenticação precisas de MACs (com chave secreta) ou assinaturas digitais (com chave privada).

Performance vs Segurança: SHA-256 é excelente balanço. SHA-512 mais seguro mas mais lento. MD5 e SHA-1 são rápidos mas inseguros - nunca uses para segurança, apenas para checksums não-críticos.

3.2. Derivação de Chaves (KDF)

Passwords humanas são o elo mais fraco. KDFs transformam passwords fracas em chaves criptográficas fortes e tornam ataques de força bruta proibitivamente caros.

Sumário de Termos

Key Derivation Function (KDF) Conceito

Função que transforma passwords ou secrets fracos em chaves criptográficas fortes através de processamento intensivo.

PBKDF2 (Password-Based Key Derivation Function 2) Algoritmo

KDF que aplica repetidamente uma função hash (ex: HMAC-SHA256) com salt para tornar ataques de força bruta mais custosos.

Salt Conceito

Valor aleatório único adicionado a passwords antes do hashing para prevenir rainbow table attacks e garantir hashes únicos.

Iteration Count (Contagem de Iterações) Parâmetro

Número de vezes que a função é aplicada repetidamente para aumentar o custo computacional de cada tentativa de ataque.

Bcrypt/Scrypt/Argon2 Algoritmos

KDFs modernas especificamente desenhadas para serem memory-hard (custosas em memória) e resistentes a hardware especializado (GPUs, ASICs).

🔗 Fundamentos Essenciais (20%)

Conceito central: Passwords humanas são fracas e previsíveis. KDFs transformam-nas em chaves criptográficas fortes aplicando milhares de iterações de hash com salt único. Isto torna ataques de força bruta extremamente lentos (ex: 100.000 iterações = 100.000x mais lento que hash simples).

Salt é obrigatório: Sem salt, passwords idênticas geram hashes idênticos, permitindo rainbow tables (tabelas pré-computadas de milhões de hashes). Com salt único por utilizador (128+ bits aleatórios), cada password gera hash diferente mesmo sendo igual. Salt não precisa ser secreto, mas deve ser único e aleatório.

Trade-off fundamental: Mais iterações = mais seguro mas mais lento. Ajusta conforme poder computacional disponível para balancear experiência do utilizador (login deve ser <1s) com segurança (atacante deve levar anos). Aumenta iterações ao longo do tempo à medida que hardware melhora.

PBKDF2 vs Bcrypt vs Argon2: PBKDF2 é mais antigo e universal mas vulnerável a GPUs. Bcrypt e Scrypt são memory-hard (precisam muita RAM, GPUs são menos eficientes). Argon2 é estado-da-arte (vencedor da Password Hashing Competition 2015), permite controlar uso de CPU e memória. Para novos sistemas, usa Argon2.

Nunca inventes KDF própria: Simplesmente concatenar salt com password e fazer hash NÃO é KDF segura. Usa implementações auditadas de PBKDF2/Bcrypt/Argon2 de bibliotecas estabelecidas.

3.3. Criptografia Simétrica Moderna

Criptografia simétrica é a espinha dorsal da segurança de dados em repouso e em trânsito. Rápida, eficiente, mas requer distribuição segura de chaves.

Sumário de Termos

Block Cipher (Cifra de Blocos) Conceito

Cifra que processa dados em blocos de tamanho fixo (ex: 128 bits) usando uma chave secreta partilhada entre emissor e recetor.

Stream Cipher (Cifra de Fluxo) Conceito

Cifra que processa dados bit-a-bit ou byte-a-byte gerando keystream, geralmente mais rápida que block ciphers.

AES (Advanced Encryption Standard) Algoritmo

Cifra de blocos simétrica (128 bits) com chaves de 128, 192 ou 256 bits, usando substituições e permutações em rounds (10/12/14).

DES/3DES Algoritmo (Legado)

Cifras legadas de 64 bits (DES inseguro com chave 56-bit, 3DES aplica 112/168 bits efetivos).

ECB (Electronic Codebook) Modo de Operação

Modo que cifra cada bloco independentemente com a mesma chave, inseguro pois revela padrões no plaintext.

CBC (Cipher Block Chaining) Modo de Operação

Modo que faz XOR de cada bloco com o bloco anterior cifrado usando um IV, escondendo padrões mas sequencial.

CTR (Counter Mode) Modo de Operação

Modo que cifra um contador incrementado e faz XOR com plaintext, permitindo paralelização e acesso aleatório.

GCM (Galois/Counter Mode) Modo Autenticado

Modo autenticado que combina CTR com autenticação usando multiplicação em Galois Field, fornecendo cifra e MAC simultaneamente.

IV (Initialization Vector) Parâmetro

Valor aleatório usado uma única vez com a chave para garantir que o mesmo plaintext produza ciphertexts diferentes.

Padding Técnica

Preenchimento adicionado ao último bloco para completar o tamanho requerido pelo block cipher (ex: PKCS#7).

🔗 Fundamentos Essenciais (20%)

Conceito central: Mesma chave cifra e decifra. Rápido (100-1000x mais que assimétrico) e eficiente para grandes volumes de dados. AES é o standard atual - extremamente seguro quando usado corretamente. O desafio é distribuir chaves de forma segura.

Modos de operação são críticos: ECB é inseguro (blocos idênticos de plaintext produzem blocos idênticos de ciphertext - padrões visíveis). CBC esconde padrões mas é sequencial (não paralelizável), CTR permite paralelização e acesso aleatório mas não tem autenticação, GCM adiciona autenticação automática. Para aplicações modernas, usa **AES-GCM**.

IV/Nonce é obrigatório: Nunca reutilizes IV com a mesma chave em CBC/CTR/GCM, senão padrões ficam visíveis ou segurança colapsa completamente. Gera IV aleatório (CBC) ou nonce único (CTR/GCM) para cada mensagem e envia-o com o ciphertext (não é secreto, pode ser transmitido em claro).

Padding attacks existem: Em modos como CBC, erros de padding podem vaziar informação (Padding Oracle Attack). GCM evita isto por ter autenticação integrada - qualquer modificação é detetada antes de decifrar.

Tamanho de chave importa: AES-128 é seguro para maioria dos casos. AES-256 oferece margem de segurança extra mas é ~40% mais lento. Para dados ultra-sensíveis ou longo prazo (10+ anos), usa AES-256. Nunca uses DES (quebrado) ou 3DES (lento e sendo descontinuado).

Hardware acceleration: CPUs modernas têm instruções AES